

BÁO CÁO CUỐI KỲ: ORBIT WARS

Nguyễn Thị Thanh Huyền
23020381

Nguyễn Thị Minh Ly
23020399

Đặng Minh Nguyệt
23020407

Tóm tắt nội dung—Báo cáo trình bày các phương pháp được nhóm nghiên cứu và thử nghiệm nhằm giải quyết bài toán Orbit Wars trên nền tảng Kaggle. Nhóm đánh giá nhiều hướng tiếp cận khác nhau bao gồm Heuristic, Heuristic kết hợp Học máy & Học tăng cường, Reward Shaping, Imitation Learning và Self-Play Training. Các phương pháp được so sánh thông qua kết quả thực nghiệm và thứ hạng trên bảng xếp hạng của cuộc thi.

I. THÔNG TIN NHÓM

A. Số thứ tự nhóm

Nhóm: IAI-RL-05

B. Danh sách thành viên

Bảng I
PHÂN CÔNG CÔNG VIỆC

Thành viên	Công việc chính
Nguyễn Thị Thanh Huyền	Heuristic và Neural Network as Validator
Nguyễn Thị Minh Ly	Imitation Learning và PPO Self-play
Đặng Minh Nguyệt	Heuristic và PPO as Validator

C. Tỷ lệ đóng góp

Bảng II
TỶ LỆ ĐÓNG GÓP

Thành viên	Tỷ lệ
Nguyễn Thị Thanh Huyền	33.(3)%
Nguyễn Thị Minh Ly	33.(3)%
Đặng Minh Nguyệt	33.(3)%

D. Thành viên đại diện nộp bài

Nguyễn Thị Minh Ly

E. Liên kết Github

<https://github.com/abcdSayHi1907/IAI-RL-05-OrbitWars>

II. GIỚI THIỆU BÀI TOÁN

A. Orbit Wars Challenge

Orbit Wars là một cuộc thi AI chiến lược nhiều tác nhân được tổ chức trên nền tảng Kaggle, lấy cảm hứng từ trò chơi Planet Wars kinh điển. Trong môi trường này, các tác nhân (agents) điều khiển hạm đội không gian nhằm chiếm đóng các hành tinh, mở rộng lãnh thổ và tối đa hóa số lượng tàu chiến sở hữu.

Mỗi trận đấu diễn ra trên bản đồ không gian hai chiều kích thước 100×100 , bao gồm các hành tinh cố định, các hành

ting quay quanh mặt trời trung tâm và các sao chổi xuất hiện theo chu kỳ. Người chơi có thể gửi hạm đội từ các hành tinh đang kiểm soát đến các mục tiêu khác để chiếm đóng hoặc tấn công đối thủ. Các hành tinh tạo ra tàu chiến theo thời gian, do đó việc cân bằng giữa mở rộng lãnh thổ, phòng thủ và tấn công đóng vai trò quan trọng trong chiến lược tổng thể.

Mỗi trận đấu kéo dài tối đa 500 lượt. Trò chơi kết thúc khi chỉ còn một người chơi tồn tại hoặc khi đạt giới hạn số lượt. Người chiến thắng là người sở hữu tổng số tàu chiến lớn nhất, bao gồm tàu trên hành tinh và tàu đang di chuyển trong không gian.

Trong cuộc thi, các agent được đánh giá thông qua hệ thống xếp hạng kỹ năng (*Skill Rating*). Các bot liên tục thi đấu với những đối thủ có trình độ tương đương và được cập nhật điểm số dựa trên kết quả thắng, thua hoặc hòa. Điều này tạo ra một môi trường đánh giá khách quan cho việc so sánh các chiến lược và mô hình học máy khác nhau.

B. Mục tiêu của bài toán

Mục tiêu của bài toán Orbit Wars là tìm kiếm một chính sách điều khiển (policy) tối ưu cho phép agent lựa chọn hành động phù hợp tại mỗi trạng thái của môi trường nhằm tối đa hóa phần thưởng tích lũy hoặc xác suất chiến thắng trong trận đấu. Trong mỗi lượt chơi, agent quan sát trạng thái hiện tại của môi trường bao gồm thông tin về hành tinh, hạm đội, sao chổi và đối thủ, sau đó quyết định cách phân bổ tàu chiến để thực hiện các hành động tấn công, phòng thủ hoặc mở rộng lãnh thổ. Chính sách tối ưu cần cân bằng giữa lợi ích ngắn hạn và dài hạn, đồng thời thích nghi với sự thay đổi liên tục của môi trường và hành vi của đối thủ.

Dưới góc độ học tăng cường (Reinforcement Learning), Orbit Wars có thể được xem là một bài toán ra quyết định tuần tự, trong đó agent tương tác với môi trường qua nhiều bước thời gian để tối đa hóa tổng phần thưởng kỳ vọng. Một chính sách hiệu quả cần học được cách khai thác tài nguyên từ các hành tinh, duy trì ưu thế quân sự và kiểm soát bản đồ nhằm đạt được kết quả cuối cùng là chiến thắng trận đấu hoặc tối đa hóa tổng số tàu chiến sở hữu khi trận đấu kết thúc.

C. Những khó khăn và thách thức

Orbit Wars là một bài toán có độ phức tạp cao do kết hợp nhiều đặc điểm thường gặp trong các môi trường học tăng cường hiện đại.

• Không gian trạng thái lớn và liên tục.

Vị trí của hành tinh, hạm đội và sao chổi được biểu diễn trong không gian liên tục, đồng thời số lượng đối tượng thay đổi theo từng thời điểm. Điều này khiến việc trích

xuất đặc trưng trạng thái trở nên khó khăn hơn so với các môi trường dạng lưới hoặc trạng thái rời rạc.

- **Không gian hành động rất lớn.**

Mỗi hành động bao gồm hành tinh xuất phát, hướng bắn và số lượng tàu được gửi đi. Sự kết hợp của các thành phần này tạo ra số lượng hành động khả thi rất lớn tại mỗi lượt.

- **Môi trường động và phi tĩnh.**

Các hành tinh có thể quay quanh mặt trời, sao chổi xuất hiện và biến mất theo thời gian, đồng thời các hạm đội liên tục di chuyển. Do đó trạng thái môi trường thay đổi ngay cả khi tác nhân không thực hiện hành động nào.

- **Bài toán nhiều tác nhân (Multi-Agent Environment).**

Hiệu quả của một chiến lược không chỉ phụ thuộc vào môi trường mà còn phụ thuộc vào hành vi của đối thủ. Khi các đối thủ thay đổi chiến thuật hoặc được cải thiện thông qua huấn luyện, phân phối dữ liệu mà mô hình quan sát được cũng thay đổi theo.

- **Phần thưởng thưa (Sparse Reward).**

Kết quả thắng hoặc thua chỉ được xác định rõ ràng ở cuối trận đấu, trong khi thời lượng một trận có thể kéo dài tới 500 lượt. Điều này gây khó khăn cho quá trình học và đòi hỏi các kỹ thuật thiết kế hàm thưởng hoặc tự chơi phù hợp.

- **Yêu cầu cân bằng giữa chiến lược ngắn hạn và dài hạn.**

Việc mở rộng quá nhanh có thể khiến hệ thống phòng thủ yếu đi, trong khi chơi quá an toàn có thể làm mất lợi thế phát triển. Tác nhân cần học được cách đánh đổi giữa lợi ích trước mắt và lợi ích dài hạn.

- **Hạn chế tài nguyên tính toán.**

Trong môi trường Kaggle, mỗi lượt chỉ có thời gian suy luận giới hạn, do đó mô hình cần đạt được sự cân bằng giữa chất lượng quyết định và tốc độ xử lý.

III. PHƯƠNG PHÁP TIẾP CẬN

A. Hướng tiếp cận Heuristic

Trong giai đoạn đầu của cuộc thi, nhóm lựa chọn hướng tiếp cận heuristic nhằm hiểu rõ cơ chế vận hành của môi trường Orbit Wars trước khi chuyển sang các phương pháp học máy và học tăng cường. Việc xây dựng một heuristic đủ mạnh để cạnh tranh với các agent dẫn đầu trên Kaggle là rất khó khăn do các agent này đã được phát triển qua nhiều phiên bản và đóng góp bởi nhiều tác giả khác nhau. Vì vậy, thay vì xây dựng từ đầu, nhóm tập trung nghiên cứu và phân tích các dòng heuristic tiêu biểu đang dẫn dắt meta của cuộc thi. Cụ thể, hai nhóm biến thể mà nhóm đã tham khảo bao gồm:

- **Nhóm World Model và Campaign Planner:** Đây là nhóm agent xây dựng một mô hình mô phỏng trạng thái tương lai của trận đấu (world model), từ đó triển khai nhiều chiến lược tấn công và phòng thủ khác nhau như giữ quân dự trữ (reserve), cứu viện (reinforcement), sơ tán khẩn cấp (doom-evac), tấn công tổng lực (hammer), tấn công đa hướng (multiprong) và duy trì kế hoạch qua nhiều lượt chơi.

- **Nhóm Tensorized Orbit-Lite (Flow-Diff Planner):** Đây là dòng agent đại diện cho meta ở giai đoạn cuối của cuộc thi và đã đánh bại nhóm agent đầu tiên. Thay vì xây dựng các chiến dịch dài hạn, agent sinh ra hàng loạt candidate action ở mỗi turn, đánh giá chúng bằng các hàm điểm dựa trên flow-difference và lựa chọn các wave có lợi nhất.

Mặc dù không trực tiếp xây dựng các heuristic từ đầu, quá trình nghiên cứu các dòng agent trên đã mang lại nhiều insight quan trọng cho nhóm.

- Nhóm World Model cho thấy lợi thế rõ rệt trong việc lập kế hoạch dài hạn. Nhờ các tầng chiến lược được xây dựng chồng lớp lên nhau, agent có thể tổ chức những chiến dịch kéo dài nhiều turn và duy trì cấu trúc chiến thuật ổn định. Tuy nhiên, chính sự phức tạp này khiến agent trở nên bảo thủ hơn, thường giữ quân dự trữ quá mức và phản ứng chậm trước các cơ hội ngắn hạn.
- Ngược lại, các agent Tensorized Orbit-Lite chứng minh rằng meta của cuộc thi đã dịch chuyển theo hướng ưu tiên local play. Thay vì tập trung vào các kế hoạch kéo dài nhiều turn, các agent này liên tục đánh giá lại trạng thái bàn cờ và khai thác các sai lầm tức thời của đối thủ. Điều này đặc biệt hiệu quả trong giai đoạn đầu trận đấu, khi việc mở rộng lãnh thổ nhanh và tận dụng tempo thường quyết định lợi thế về sau. Mặc dù vượt trội so với nhóm agent thể hệ đầu, hướng tiếp cận này vẫn thiên về tối ưu cục bộ và ra quyết định mang tính tham lam. Khả năng phòng thủ chủ yếu dựa trên khả năng phản ứng thay vì kế hoạch dài hạn, khiến đây khó có thể là một hướng tiếp cận ổn định nếu cuộc thi tiếp tục kéo dài.

B. Heuristic kết hợp Học máy & Học tăng cường

Một trong những nhận định quan trọng của nhóm là dù heuristic được thiết kế tinh vi đến đâu vẫn khó tránh khỏi các quyết định chưa tối ưu trong những tình huống cụ thể. Vì vậy, nhóm nghiên cứu hướng tiếp cận sử dụng học máy & học tăng cường như một validator, với vai trò đánh giá và xếp hạng lại các hành động do heuristic sinh ra, từ đó lựa chọn phương án có xác suất thành công cao nhất.

1) Mạng nơ-ron:

a) **Hướng 1 - Validator xác suất thành công với top-k throttle:** Hướng tiếp cận đầu tiên được tham khảo từ notebook [1]. Ý tưởng chính là sử dụng một heuristic mạnh để sinh các hành động ứng viên, sau đó huấn luyện một mạng MLP nhằm dự đoán xác suất thành công của từng hành động. xác suất này thể hiện khả năng hành động đó giúp agent chiếm được hành tinh mục tiêu sau khi tàu đến nơi.

Quy trình tổng thể bao gồm: thu thập dữ liệu thông qua self-play với một tập đối thủ đa dạng, trích xuất các shot từ replay, xây dựng tập đặc trưng cho từng hành động, gán nhãn kết quả sau khi hành động được thực thi, huấn luyện mạng MLP và cuối cùng tích hợp mô hình vào heuristic dưới dạng một validator. Một shot được gán nhãn thành công nếu agent sở hữu hành tinh mục tiêu trong một khoảng thời gian sau thời điểm đội tàu đến nơi.

Trong quá trình tái hiện phương pháp này, nhóm thực hiện một số cải tiến chính:

- Xây dựng lại pipeline thu thập dữ liệu và self-play với tập đối thủ lớn hơn nhằm tăng độ đa dạng của dữ liệu huấn luyện.
- Điều chỉnh tập đặc trưng đầu vào. So với 24 đặc trưng gốc, nhóm bổ sung thêm 5 đặc trưng mới và loại bỏ 3 đặc trưng ít đóng góp, giúp độ chính xác của validator tăng từ khoảng 75% lên 81%.
- Thực hiện các thí nghiệm với nhiều cấu hình top- k khác nhau (top-1, top-2, top-3) để đánh giá ảnh hưởng của cơ chế lọc hành động và lựa chọn cấu hình phù hợp nhất.

b) Hướng 2 - Validator rủi ro và xếp hạng lại hành động:

Mặc dù Hướng 1 giúp loại bỏ các shot khó chiếm mục tiêu, tiêu chí huấn luyện vẫn chủ yếu tập trung vào kết quả chiếm mục tiêu. Trong thực tế, một shot có thể chiếm được mục tiêu nhưng vẫn là quyết định rủi ro nếu làm mất hành tinh nguồn hoặc khiến mục tiêu bị chiếm lại ngay sau đó. Vì vậy, nhóm tiếp tục thử nghiệm Hướng 2, trong đó validator học trực tiếp xác suất rủi ro chiến thuật của hành động.

Ở hướng này, heuristic gốc vẫn chịu trách nhiệm sinh các hành động ứng viên. Mạng nơ-ron không được dùng để sinh hành động mới, mà đóng vai trò đánh giá lại các hành động tấn công do heuristic đề xuất. Đầu ra của mô hình là xác suất một hành động thuộc nhóm rủi ro cao, tức khả năng hành động đó dẫn đến một kết quả bất lợi trong ngắn hạn.

Mỗi mẫu huấn luyện tương ứng với một hành động tấn công. Nhãn của hành động được xác định dựa trên hậu quả sau khi hành động được thực thi. Một hành động được xem là tệ nếu rơi vào một trong ba trường hợp sau: (1) Không chiếm được mục tiêu sau khi đội tàu đến nơi. (2) Làm mất hành tinh nguồn trong thời gian ngắn sau khi phóng quân. (3) Chiếm được mục tiêu nhưng bị đối thủ chiếm lại quá nhanh. Cách gán nhãn này phản ánh đầy đủ hơn tính an toàn của quyết định tấn công so với việc chỉ kiểm tra riêng khả năng chiếm được mục tiêu.

So với Hướng 1, nhóm điều chỉnh tập đặc trưng còn 20 đặc trưng tập trung hơn vào rủi ro chiến thuật, bao gồm điểm đánh giá của heuristic, mức độ rút quân khỏi nguồn, biên độ chiếm mục tiêu và áp lực địch quanh nguồn/mục tiêu. Mạng MLP nhỏ sử dụng các đặc trưng này để dự đoán xác suất hành động thuộc nhóm rủi ro cao.

Tập dữ liệu cuối cùng gồm khoảng 16 nghìn mẫu hành động, được thu thập từ các trận đấu giữa agent nền với nhiều đối thủ khác nhau và từ cơ chế self-play. Trên tập validation, mô hình đạt AUC xấp xỉ 0.84. Chỉ số AUC (Area Under the ROC Curve) được sử dụng vì validator trong hướng tiếp cận này chủ yếu đóng vai trò xếp hạng mức độ rủi ro giữa các hành động thay vì phân loại theo một ngưỡng cố định, do đó phản ánh tốt khả năng phân biệt và xếp hạng các hành động an toàn.

Khi triển khai, validator được tích hợp như một bước hậu xử lý đầu ra. Heuristic gốc sinh danh sách hành động như bình thường; các hành động hỗ trợ hoặc phòng thủ được giữ lại, trong khi các hành động tấn công được đưa qua mạng MLP để ước lượng xác suất rủi ro. Sau đó, agent chỉ giữ tối đa hai hành động tấn công có xác suất rủi ro thấp nhất. Cách tích hợp này giúp giảm các đợt tấn công kém chất lượng mà không làm thay đổi hoàn toàn hành vi chiến thuật của heuristic.

Tóm lại, điểm khác biệt chính của Hướng 2 là validator không còn chỉ đánh giá khả năng chiếm mục tiêu của từng shot, mà đánh giá rủi ro chiến thuật tổng thể của hành động. Điều này giúp mô hình xử lý tốt hơn các trường hợp một cú tấn công có vẻ hợp lệ trong ngắn hạn nhưng gây bất lợi cho thế trận sau đó.

2) *Proximal Policy Optimization*: Một trong những khó khăn lớn của việc áp dụng Reinforcement Learning vào Orbit Wars là không gian hành động rất lớn. Nếu để PPO tự học toàn bộ luật chơi, cơ chế vật lý và cách tạo từng lệnh tấn công từ đầu, lượng dữ liệu và thời gian huấn luyện cần thiết sẽ rất lớn. Vì vậy, thay vì để PPO trực tiếp sinh ra các hành động, nhóm lựa chọn một hướng tiếp cận kết hợp giữa heuristic và học tăng cường. Ý tưởng cốt lõi là sử dụng heuristic như một *teacher* để sinh ra nhiều kế hoạch hợp lệ, sau đó PPO học cách lựa chọn kế hoạch tốt nhất trong số đó. Nói cách khác, heuristic chịu trách nhiệm tạo phương án, còn PPO chịu trách nhiệm ra quyết định.

Quy trình tổng thể được mô tả trong Hình 1. Đầu tiên, trạng thái hiện tại của trận đấu được mã hóa thành các đặc trưng đầu vào. Đồng thời, các biến thể heuristic sinh ra tập các candidate plans. Mỗi kế hoạch được trích xuất đặc trưng riêng và kết hợp với biểu diễn của trạng thái hiện tại trước khi đưa vào mạng chính sách (*policy network*). Mạng này dự đoán xác suất cho từng kế hoạch và lựa chọn một phương án để thực thi trong môi trường.

a) *Tạo action candidates*: Tại mỗi lượt chơi, cùng một trạng thái của môi trường sẽ được đưa qua nhiều biến thể heuristic khác nhau, bao gồm: `pass`, `baseline`, `aggressive`, `very_aggressive`, `conservative`, `no_regroup`, `risk_regroup`, `short_horizon`, `long_horizon` và `no_focus_fire`. Mỗi heuristic lại sinh ra một kế hoạch hoàn chỉnh gồm nhiều hành động phóng quân dưới dạng:

$[[source_planet_id, angle, ships], \dots]$

Do đó, thay vì học trong không gian hành động khổng lồ của Orbit Wars, PPO chỉ cần lựa chọn một kế hoạch tối ưu trong tập các phương án do heuristic đề xuất, giúp giảm đáng kể độ khó của bài toán học tăng cường.

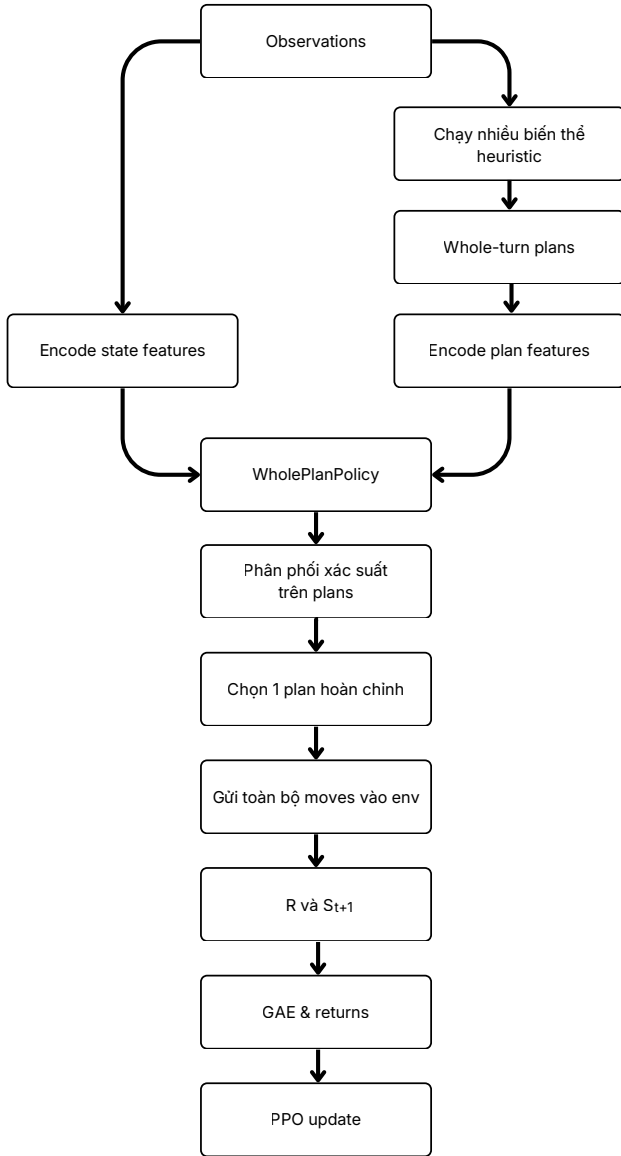
b) *Biểu diễn trạng thái*: Mạng PPO sử dụng bốn nhóm đặc trưng chính (các state features được tham khảo từ notebook [2]):

- Planet features.
- Fleet features.
- Global game features.
- Summary features của từng candidate plan.

c) *Lựa chọn kế hoạch*: Sau khi thu được biểu diễn của trạng thái và các candidate plans, mạng chính sách sẽ tính điểm cho từng kế hoạch: $\logit_i = f(\text{state}, \text{plan}_i)$. Các điểm số này được chuyển thành phân phối xác suất thông qua hàm softmax.

Trong giai đoạn huấn luyện, hành động được lấy mẫu từ phân phối xác suất: $a \sim \pi(a|s)$.

Trong khi đó, ở giai đoạn đánh giá hoặc submission, agent luôn chọn greedy: $a = \arg \max_i \pi(a_i|s)$.



Hình 1. Quy trình phương pháp PPO as Validator

d) *Thiết kế phần thưởng*: Ngoài phần thưởng cuối trận (thắng hoặc thua), nhóm sử dụng thêm *potential-based reward shaping* để cung cấp tín hiệu học dày đặc hơn trong quá trình huấn luyện. Các thành phần shaping được xây dựng từ sự thay đổi của:

- Chênh lệch số lượng hành tinh.
- Chênh lệch năng suất sản xuất quân.
- Chênh lệch tổng số tàu.

e) *Thu thập dữ liệu và cập nhật PPO*: Trong quá trình huấn luyện, nhiều môi trường được chạy song song để thu thập các rollout. Với mỗi bước thời gian, hệ thống lưu lại:

- Trạng thái hiện tại.

- Chỉ số kế hoạch được chọn.
- Log-probability của hành động.
- Giá trị dự đoán bởi value network.
- Reward nhận được.
- Trạng thái kết thúc trận đấu (nếu có).

Sau khi hoàn thành một rollout, nhóm sử dụng Generalized Advantage Estimation (GAE) để tính advantage: A_t^{GAE} , và tính giá trị mục tiêu: $R_t = A_t^{GAE} + V(s_t)$. Các giá trị này được sử dụng để cập nhật policy và value network theo thuật toán PPO.

Hàm mất mát cuối cùng bao gồm ba thành phần chính:

- Clipped policy loss.
- Value loss.
- Entropy bonus.

Các chỉ số được theo dõi trong quá trình học bao gồm explained variance, approximate KL divergence, clip fraction, entropy và tỷ lệ lựa chọn hành động pass.

C. Imitation Learning kết hợp PPO Self-Play

Hướng tiếp cận cuối cùng của nhóm là kết hợp giữa heuristic, học bắt chước (Imitation Learning) và học tăng cường (Reinforcement Learning). Ý tưởng cốt lõi là tận dụng heuristic như một nguồn tri thức ban đầu để giảm độ khó của không gian hành động, sau đó sử dụng mạng nơ-ron và PPO để học cách ra quyết định thông qua quá trình tự chơi.

a) *Thu thập dữ liệu bằng heuristic expert*: Một heuristic bot đóng vai trò expert policy. Bot này sử dụng các luật thủ công để đánh giá trạng thái trò chơi dựa trên các yếu tố như khoảng cách, sản lượng hành tinh, số lượng tàu và mức độ rủi ro, từ đó lựa chọn hành động phù hợp. Để xây dựng tập dữ liệu huấn luyện, nhóm cho hai heuristic bot tự đấu với nhau. Trong quá trình thi đấu, tại mỗi trạng thái, hệ thống ghi lại hành động được heuristic lựa chọn.

Bên cạnh đó, một tập các hành động ứng viên (candidate actions) cũng được sinh ra, bao gồm các chiến thuật phổ biến như:

- Mở rộng sang hành tinh trung lập.
- Tấn công hành tinh đối phương.
- Phòng thủ hành tinh đang bị đe dọa.
- Tiếp viện giữa các hành tinh đồng minh.
- Tập trung lực lượng (regroup).

Hành động ứng viên giống với quyết định của heuristic nhất sẽ được sử dụng làm nhãn huấn luyện.

b) *Biểu diễn trạng thái và hành động*: Mỗi observation được chuyển thành tập đặc trưng mô tả trạng thái hiện tại của trò chơi, bao gồm thông tin về hành tinh, fleet và các thống kê toàn cục. Đối với mỗi candidate action, nhóm cũng xây dựng một vector đặc trưng bao gồm:

- Hành tinh nguồn.
- Hành tinh đích.
- Số lượng tàu được gửi.
- Loại hành động.
- Kết quả dự đoán từ một mô phỏng ngắn trong tương lai.

c) *Behavior Cloning*: Mô hình sử dụng kiến trúc Transformer để tổng hợp thông tin trạng thái toàn cục. Mỗi candidate action được mã hóa riêng, sau đó kết hợp với biểu diễn của trạng thái trò chơi. Nhiệm vụ của mạng là dự đoán candidate nào sẽ được heuristic lựa chọn. Quá trình này được thực hiện bằng học có giám sát (supervised learning). Sau khi hoàn thành giai đoạn Behavior Cloning, mô hình thu được một chính sách ban đầu có khả năng mô phỏng tương đối chính xác hành vi của heuristic expert.

d) *PPO Self-Play*: Sau khi hoàn thành Behavior Cloning, trọng số của mạng được sử dụng để khởi tạo PPO. Trong quá trình huấn luyện, agent liên tục tự chơi với các phiên bản trước đó của chính mình. Một checkpoint pool được duy trì nhằm lưu trữ nhiều phiên bản agent ở các mức Elo khác nhau. Đối thủ trong mỗi trận đấu được lấy mẫu từ pool này nhằm tăng tính đa dạng của dữ liệu huấn luyện và giảm hiện tượng overfitting vào một chiến lược cụ thể.

Tại mỗi bước thời gian, hệ thống lưu lại:

- Trạng thái hiện tại.
- Candidate được chọn.
- Log-probability của hành động.
- Giá trị trạng thái.
- Phần thưởng nhận được.

Sau khi hoàn thành rollout, nhóm sử dụng Generalized Advantage Estimation (GAE) để tính advantage và cập nhật policy theo thuật toán PPO.

e) *Reward Shaping*: Để cung cấp tín hiệu học dày đặc hơn thay vì chỉ sử dụng kết quả thắng thua cuối trận, nhóm bổ sung cơ chế reward shaping dựa trên sự thay đổi tương đối giữa hai người chơi.

Các thành phần được sử dụng bao gồm:

- Chênh lệch số lượng hành tinh kiểm soát.
- Chênh lệch tổng năng suất sản xuất.
- Chênh lệch tổng số lượng tàu.

Thiết kế này giúp agent học được các hành vi mở rộng lãnh thổ, kiểm soát tài nguyên và duy trì ưu thế quân sự ngay từ giai đoạn đầu trận đấu.

f) *Điểm khác biệt cốt lõi*: Khác với hướng PPO as Validator đã trình bày ở trên, heuristic trong phương pháp này chỉ được sử dụng ở giai đoạn thu thập dữ liệu và huấn luyện Behavior Cloning. Mô hình học cách bắt chước các hành động của heuristic expert để hình thành một chính sách khởi tạo hợp lý. PPO sau đó tiếp tục tối ưu chính sách này thông qua self-play, cho phép agent dần rời khỏi hành vi bắt chước thuần túy và khám phá những quyết định có thể mang lại phần thưởng cao hơn.

Trong khi đó, ở hướng PPO as Validator, heuristic vẫn đóng vai trò trung tâm trong quá trình suy luận. PPO không trực tiếp lựa chọn hành động mà chỉ lựa chọn một kế hoạch (plan) hoặc một biến thể heuristic phù hợp nhất, còn toàn bộ quá trình thực thi chi tiết vẫn được quyết định bởi heuristic. PPO đóng vai trò bộ điều phối cấp cao (high-level planner), còn heuristic chịu trách nhiệm triển khai chiến thuật cụ thể.

IV. THỰC NGHIỆM

A. Các phiên bản heuristic

1) *Nỗ lực cải tiến World Model Agent*: Ban đầu, nhóm lựa chọn cải tiến một agent thuộc dòng World Model là h3b1 [3] với mục tiêu tăng cường khả năng local play để cạnh tranh với các agent Tensorized Orbit-Lite. Tuy nhiên, trong quá trình thực nghiệm, nhóm nhận thấy các thành phần của h3b1 được liên kết rất chặt chẽ với nhau. Các module như reserve planner, commitment manager, multiprong planner, hammer planner và Melis search sử dụng các cơ chế đánh giá khác nhau nhưng lại ảnh hưởng lẫn nhau trong quá trình ra quyết định. Điều này khiến việc cải thiện một thành phần riêng lẻ thường không mang lại hiệu quả tổng thể.

Nhóm đã thử nghiệm nhiều hướng cải tiến:

- 1) **Sửa độ chính xác của mô phỏng**: kết quả 0/10 wins
- 2) **Tăng nhịp độ đầu trận**: trì hoãn depth-2 search trong opening, giảm ngưỡng sanity check và tăng quota hành động ở chế độ 2-player nhằm khuyến khích agent mở rộng sớm hơn. Tuy nhiên kết quả vẫn là 0/10 wins. Mặc dù số lượng candidate action tăng lên, các tầng reserve, safety và target-lock vẫn tiếp tục chặn lẫn nhau, khiến agent không thực sự hành động nhiều hơn.
- 3) **Quản lý commitment**: bổ sung cơ chế lưu và loại bỏ các commitment trùng lặp, giới hạn ETA khi đánh neutral và loại bỏ các commitment đã hết hiệu lực. Đây là cải tiến đầu tiên mang lại kết quả tích cực với tỷ lệ 1 thắng - 9 thua, nhưng vẫn kém xa các agent Tensorized Orbit-Lite.

Vấn đề của h3b1 không nằm ở một lỗi đơn lẻ. Các module chiến lược sử dụng những mô hình đánh giá khác nhau và đôi khi mâu thuẫn với nhau, khiến agent duy trì tỷ lệ idle cao và có xu hướng giữ quân quá mức. Trung bình agent chỉ thực hiện khoảng 0.47 hành động mỗi turn, với tỷ lệ idle xấp xỉ 64.8%, dẫn đến việc bỏ lỡ tempo mở rộng ở giai đoạn đầu trận.

2) *Chuyển hướng sang Tensorized Orbit-Lite*: Sau khi nhận thấy những hạn chế của dòng World Model, nhóm quyết định chuyển sang nền tảng Tensorized Orbit-Lite ở giai đoạn cuối của cuộc thi. Dựa trên agent buddy, nhóm đề xuất phiên bản buddy_v12 với một thay đổi kiến trúc đơn giản nhưng hiệu quả. Trong phiên bản gốc, agent ưu tiên các mục tiêu gần nhất khi xây dựng shortlist. Trong buddy_v12, ở 40 turn đầu tiên, điểm số mục tiêu được bổ sung thêm thành phần phản ánh năng suất sản xuất quân (production weight). Thay đổi này cho phép agent chấp nhận các mục tiêu xa hơn nếu chúng mang lại lợi ích kinh tế lớn hơn, tăng lợi thế sản xuất từ sớm cũng như phá vỡ các trạng thái đối xứng giữa 2 agent.

Kết quả thực nghiệm cho thấy buddy_v12 đạt 8/10 trận có reward dương trong tập đánh giá đầu tiên. Trên một tập xác nhận độc lập gồm 20 trận với các seed khác nhau và hoán đổi vị trí xuất phát, agent đạt 12 trận reward dương và 8 trận reward âm, với trung bình 276.1 bước mỗi trận, 1.422 hành động mỗi turn và tỷ lệ idle chỉ còn 34.2%.

B. Các phiên bản validator

1) *Mạng nơ-ron đánh giá xác suất thành công với top-k throttle*:

topk1: W-L-D=37-23-0 win_rate=0.617 avg_steps=150.5 moves/t=0.610 idle=0.390 invalid=0 errors=0 timeouts=0
topk2: W-L-D=39-21-0 win_rate=0.650 avg_steps=162.9 moves/t=0.887 idle=0.431 invalid=0 errors=0 timeouts=0
topk3: W-L-D=34-26-0 win_rate=0.567 avg_steps=156.7 moves/t=1.009 idle=0.440 invalid=0 errors=0 timeouts=0

Hình 2. Kết quả thực nghiệm với các ngưỡng top-k throttle khác nhau

submission.tar.gz	1194.6
Complete - CurMinCut - 9h ago - orbbite heuristic + ml validator	
ver12.py	1156.1
Complete - CurMinCut - 10h ago	

Hình 3. Kết quả leaderboard so sánh giữa có và không có validator trên cùng baseline buddy_v12

a) *Thực nghiệm với top-k throttle:* Để đánh giá ảnh hưởng của tham số top-k, nhóm thực hiện các trận đấu đối kháng giữa agent sử dụng validator và agent baseline. Mỗi cấu hình được đánh giá trên 16 seed khác nhau, đồng thời hoán đổi vị trí xuất phát giữa hai agent, tạo thành tổng cộng 32 trận đấu cho mỗi giá trị k .

Mặc dù các cấu hình top-1, top-2 và top-3 cho tỷ lệ thắng khá tương đồng, kết quả cho thấy việc sử dụng validator đã đủ để loại bỏ phần lớn các hành động chất lượng thấp. Do đó, sự khác biệt giữa các giá trị k chủ yếu nằm ở tốc độ kết thúc trận đấu thay vì tỷ lệ chiến thắng cuối cùng. Trong các thí nghiệm, cấu hình top-2 đạt thời lượng trận đấu ngắn hơn trong khi vẫn duy trì tỷ lệ thắng tương đương (Hình 2), nên được lựa chọn cho bản final.

b) *Kết quả:* Hình 3 cho thấy việc bổ sung mạng nơ-ron validator giúp agent tăng khoảng 38 điểm xếp hạng trên leaderboard sau 9 giờ đánh giá. Đây là một mức cải thiện đáng kể nếu xét rằng toàn bộ hệ thống chiến thuật cốt lõi vẫn được giữ nguyên, chỉ bổ sung thêm một tầng đánh giá hành động.

Như vậy, validator đóng vai trò như một cơ chế sửa lỗi, sử dụng các mẫu đã học được từ dữ liệu self-play để ước lượng xác suất thành công của từng hành động. Thay vì thực thi trực tiếp hành động có điểm heuristic cao nhất, agent ưu tiên những hành động vừa có điểm heuristic tốt vừa được mạng nơ-ron đánh giá tích cực. Điều này giúp giảm các trường hợp overcommit, tấn công vào mục tiêu kém hiệu quả hoặc thực hiện các pha trao đổi bất lợi.

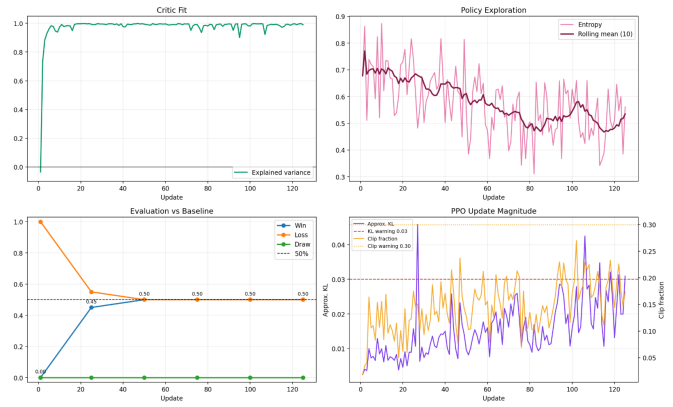
2) *Mạng nơ-ron đánh giá rủi ro và xếp hạng lại hành động:* Thiết lập đánh giá được giữ tương tự phần trước, với mở rộng thêm tập đối thủ đa dạng để kiểm tra khả năng tổng quát.

a) *Chiến lược tích hợp validator:* Nhóm so sánh hai cách tích hợp validator trong pipeline.

- Cách thứ nhất can thiệp trực tiếp trong candidate-space, khi validator điều chỉnh điểm heuristic trước bước chọn hành động cuối cùng.
- Cách thứ hai chỉ áp dụng ở final-move-space, trong đó validator đóng vai trò xếp hạng lại các hành động tấn công và giữ lại top-k hành động an toàn nhất.

Kết quả so sánh: Thực nghiệm cho thấy cách tiếp cận hậu xử lý (final-move-space) cho kết quả ổn định và tốt hơn. Cách tiếp cận này giữ nguyên cấu trúc chiến thuật của heuristic gốc trong khi vẫn loại bỏ được các hành động có rủi ro cao.

b) *Cơ chế sử dụng điểm dự đoán:* Ngoài ra, nhóm so sánh giữa hai cách sử dụng output của validator: threshold



Hình 4. Biểu đồ thể hiện các chỉ số đánh giá PPO

cứng và ranking tương đối. Kết quả cho thấy ranking ổn định hơn do phân phối xác suất thay đổi theo trạng thái trận đấu, vì vậy phiên bản cuối cùng sử dụng cơ chế xếp hạng thay vì ngưỡng cố định.

c) *Kết quả:* Thực nghiệm cho thấy phiên bản sử dụng validator theo cơ chế hậu xử lý vẫn cải thiện hiệu năng so với baseline, nhờ khả năng loại bỏ các hành động tấn công có rủi ro cao trong quá trình ra quyết định. Tuy nhiên, mức cải thiện này không đồng đều trên các đối thủ và phụ thuộc nhiều vào cấu trúc chiến thuật của heuristic gốc.

Kết luận về hai chiến lược sử dụng mạng nơ-ron làm validator: Dựa trên kết quả thực nghiệm và điểm nộp lên hệ thống, Hướng 1 cho hiệu năng tốt hơn Hướng 2 trên các thiết lập đối kháng. Sự khác biệt này chủ yếu xuất phát từ mục tiêu học của hai hướng: Hướng 1 ước lượng xác suất thành công của hành động, phù hợp trực tiếp với mục tiêu tối ưu của trò chơi, giúp tín hiệu học ổn định và dễ tối ưu hơn.

Trong khi đó, Hướng 2 mở rộng sang ước lượng rủi ro chiến thuật dài hạn, bao gồm các hậu quả gián tiếp như mất hành tinh nguồn hoặc bị tái chiếm. Mặc dù cách tiếp cận này giúp cải thiện mức độ an toàn trong một số tình huống, nó đồng thời làm giảm khả năng khai thác các hành động có lợi nhưng mang tính biên, dẫn đến hiệu năng tổng thể thấp hơn so với Hướng 1.

3) *Proximal Policy Optimization:*

a) *Kết quả huấn luyện PPO:* Hình 4 cho ta cái nhìn tổng quan về chất lượng huấn luyện PPO.

Critic fit. Explained variance tăng rất nhanh, từ giá trị âm lên khoảng 0.9 chỉ sau vài update đầu tiên, sau đó duy trì ở mức rất cao, khoảng 0.98-1.0, cho thấy critic học được hàm return khá tốt. Tuy nhiên, mức explained variance gần 1 khá bất thường trong khi kết quả trận đấu vẫn có nhiều. Có thể critic chủ yếu học tốt phần shaping reward và bootstrap return, vốn dễ dự đoán hơn terminal outcome.

Policy exploration. Entropy ban đầu khoảng 0.7 và giảm dần về vùng 0.45-0.55. Mặc dù có dao động, entropy không rơi về gần 0, cho thấy policy bắt đầu hình thành preference rõ ràng hơn nhưng chưa bị collapse. Đến update 125, agent vẫn duy trì được một mức độ khám phá nhất định.

	opponent	games	wins	losses	draws	win_rate	avg_reward	avg_steps	avg_moves_per_turn	avg_idle_rate
0	h3b1	20	14	6	0	0.70	0.4	151.40	1.564053	0.244384
1	multi-focus-new	20	3	17	0	0.15	-0.7	127.05	1.304574	0.268095
2	top_now	20	12	8	0	0.60	0.2	148.15	1.723880	0.178369
3	ver12	20	3	17	0	0.15	-0.7	123.40	1.303704	0.283643

Hình 5. Kết quả đánh giá PPO trên các đối thủ trong môi trường local

	whole_plan_ppo.zip	1046.1	
Complete • CudaMinCut • 2d ago			

Hình 6. Kết quả đánh giá PPO as Validator trên leaderboard

Evaluation vs baseline. Win rate tăng nhanh từ 0% ở update 1 lên 45% ở update 25 và đạt 50% ở update 50. Tuy nhiên, từ update 50 đến update 125, win rate gần như giữ nguyên ở mức 50%. Điều này cho thấy PPO học rất nhanh để bắt kịp heuristic baseline, nhưng sau đó rơi vào trạng thái plateau. Chưa có bằng chứng rõ ràng rằng PPO vượt trội hơn baseline.

PPO update magnitude. KL divergence có xu hướng tăng dần theo thời gian, với một số update vượt ngưỡng cảnh báo 0.03 và giá trị lớn nhất đạt 0.046. Clip fraction cũng tăng tương ứng nhưng vẫn dưới 0.30, với giá trị tối đa khoảng 0.27. Policy vẫn tiếp tục thay đổi tương đối mạnh, dù hiệu năng đánh giá không còn cải thiện. Nhiều khả năng optimizer đang dịch chuyển policy trong một vùng chiến thuật tương đương, thay vì tìm được chiến lược thực sự tốt hơn.

Như vậy, PPO đã học được cách lựa chọn plan đủ tốt để đạt hiệu năng ngang với baseline, nhưng chưa vượt qua được baseline. Giới hạn chính có thể nằm ở action library: policy hiện chỉ được chọn giữa một số preset heuristic cố định, nên không có đủ không gian để khám phá các chiến lược mới. Vì vậy, việc tiếp tục huấn luyện lâu hơn với cùng cấu hình khó tạo ra cải thiện đáng kể. Để vượt baseline rõ rệt, cần bổ sung các plan variant thực sự khác biệt hoặc cho phép PPO điều chỉnh trực tiếp các tham số chiến thuật như `roi_threshold`, `horizon`, `risk_weight` và `ship_sizing`, thay vì chỉ chọn giữa các preset có sẵn.

b) *Kết quả đấu trận & trên leaderboard:* Kết quả từ đấu trận và trên leaderboard cho kết luận tương tự: giới hạn hiệu năng của action library hiện tại khiến PPO chưa thể tạo ra sự cải thiện đáng kể về tỷ lệ thắng hay thứ hạng trên leaderboard.

C. Heuristic Expert và Behavioral Cloning Pipeline

1) *Chất lượng dữ liệu Behavioral Cloning:* Kết quả oracle coverage test trên 200 states cho thấy chất lượng dữ liệu BC còn hạn chế. Tỷ lệ heuristic action trùng với oracle chỉ đạt khoảng 22%. Ngoài ra, tỷ lệ heuristic nằm trong top-3 action cũng ở mức thấp (<60%).

Kết quả này cho thấy HeuristicBot có xu hướng chọn hành động không ổn định trong nhiều trạng thái. Thống kê hành vi chỉ ra rằng khoảng 78% state có sự sai lệch giữa lựa chọn của heuristic và oracle, dẫn đến việc dữ liệu BC chứa nhiều label nhiễu.

Điều này khiến mô hình BC học theo một policy không tối ưu ngay từ đầu, làm giảm chất lượng biểu diễn hành vi và ảnh hưởng trực tiếp đến khả năng khởi tạo cho PPO.

2) *Label bias trong tập dữ liệu BC:* Quan sát phân phối nhãn cho thấy hiện tượng label bias rõ rệt. Xác suất $P(label = 0)$ cao bất thường, do heuristic bot thường ưu tiên candidate đầu tiên trong danh sách.

Nguyên nhân chính đến từ cơ chế sinh candidate: danh sách hành động được sắp xếp theo số lượng ships giảm dần, khiến các hành động có quy mô lớn thường xuất hiện ở vị trí đầu tiên. Điều này trùng với xu hướng lựa chọn của heuristic, dẫn đến hiện tượng “position bias” trong dữ liệu.

Hệ quả là mô hình BC có xu hướng hội tụ về một hành vi đơn điệu, ưu tiên các hành động có ships lớn mà không phân biệt rõ chất lượng chiến lược. Điều này làm giảm diversity của policy và hạn chế khả năng tổng quát hóa.

3) *Cải tiến heuristic và tác động thực nghiệm:* Nhóm tiến hành cải tiến HeuristicBot và candidate generation bằng cách bổ sung cơ chế ROI-based scoring, kết hợp production và travel cost. Ngoài ra, cơ chế phòng thủ được thêm vào để phát hiện các fleet địch đang tiến tới planet của người chơi.

Bên cạnh đó, không gian hành động được mở rộng với 5 loại chiến lược gồm expand, attack, defense, reinforce và hammer, nhằm tăng độ bao phủ của policy.

Sau khi cải tiến, oracle coverage được cải thiện từ 22% lên khoảng trên 35%. Điều này cho thấy heuristic đã giảm được một phần bias nhưng vẫn còn hạn chế trong việc mô hình hóa đầy đủ chiến lược tối ưu.

4) *Tổng hợp nguyên nhân thực nghiệm:* Từ các kết quả quan sát được, có thể rút ra chuỗi nguyên nhân chính dẫn đến hiệu năng chưa đạt kỳ vọng. Thứ nhất, chất lượng dữ liệu BC thấp do heuristic bot chỉ đạt khoảng 22% oracle match, khiến mô hình học từ tín hiệu nhiễu. Thứ hai, hiện tượng label bias trong candidate generation làm giảm diversity của policy. Thứ ba, các sai lệch này lan truyền sang giai đoạn PPO self-play, khiến quá trình tối ưu không thể vượt qua được baseline heuristic.

V. KẾT LUẬN

Báo cáo đã khảo sát và thực nghiệm nhiều hướng tiếp cận khác nhau cho bài toán Orbit Wars, từ các phương pháp heuristic đến các kỹ thuật học máy và học tăng cường. Thông qua quá trình nghiên cứu và đánh giá thực nghiệm, nhóm nhận thấy việc kết hợp giữa heuristic và các mô hình mang lại hiệu quả tốt hơn so với việc sử dụng riêng lẻ từng thành phần. Trong quá trình thực hiện, nhóm không chỉ xây dựng được các phiên bản agent hoạt động hiệu quả mà còn tích lũy thêm nhiều kiến thức thực tiễn về thiết kế heuristic, học bắt chước và học tăng cường trong môi trường đa tác tử.

Tuy nhiên, bài toán Orbit Wars cũng là một thử thách lớn với toàn bộ các thành viên trong nhóm. Nhóm chưa có khả năng tự xây dựng một heuristic có khả năng cạnh tranh với các agent hàng đầu trên leaderboard. Bên cạnh đó, quá trình huấn luyện Reinforcement Learning cũng gặp nhiều trở ngại liên quan đến thiết kế môi trường, reward shaping, self-play và các chi tiết triển khai, nên nhóm chưa thực sự đạt được mục tiêu ban đầu là triển khai thuật toán RL từ đầu.

Qua quá trình thực hiện đề tài, nhóm nhận thấy vẫn còn nhiều hướng tiếp cận tiềm năng chưa có điều kiện triển khai đầy đủ. Trong tương lai, nhóm mong muốn tiếp tục tìm hiểu

sâu hơn về học tăng cường, đặc biệt là các phương pháp có khả năng tự học chiến lược mà không cần phụ thuộc quá nhiều vào heuristic. Ngoài ra, việc sử dụng Transformer để học từ lịch sử hành động của các agent mạnh, cũng như các hướng tiếp cận như self-play quy mô lớn, world model hay AlphaZero-style planning, đều là những ý tưởng mà nhóm mong muốn được thử nghiệm tiếp theo.

TÀI LIỆU

- [1] konbu17, “Train & submit v4 ml validator top-k2 tutorial,” <https://www.kaggle.com/code/konbu17/train-submit-v4-ml-validator-topk2-tutorial>, 2025, kaggle Notebook.
- [2] kashiwaba, “Orbit wars reinforcement learning tutorial,” <https://www.kaggle.com/code/kashiwaba/orbit-wars-reinforcement-learning-tutorial>, 2025, kaggle Notebook.
- [3] stealthtechnologies, “[lb 1050+] heuristic simulation agent test 3,” <https://www.kaggle.com/code/stealthtechnologies/lb-1050-heuristic-simulation-agent-test-3>, 2025, kaggle Notebook.